

Nama : Carlos Fadellilah Rama

Kelas : TI-2E

Matkul : PBO

Tujuan Utama Enkapsulasi

Enkapsulasi bertujuan untuk **menyembunyikan detail internal (data) suatu objek dari akses langsung pihak luar**, dan hanya mengizinkan interaksi melalui method yang sudah disediakan. Manfaatnya:

1. **Melindungi integritas data** — mencegah nilai atribut diubah sembarangan sehingga jadi tidak valid.
2. **Menyembunyikan kompleksitas implementasi** — pengguna class cukup tahu *apa* yang bisa dilakukan, tanpa perlu tahu *bagaimana* caranya.
3. **Memudahkan validasi** — perubahan data bisa difilter/divalidasi lewat method sebelum benar-benar disimpan.
4. **Meningkatkan maintainability** — implementasi internal bisa diubah tanpa mempengaruhi kode lain yang menggunakan class tersebut.

Contoh Kasus: Objek RekeningBank

Rekening bank adalah contoh klasik karena saldo tidak boleh diubah sembarangan dari luar — harus lewat mekanisme resmi (setor/tarik).

```
public class RekeningBank {  
    // 1. Atribut private, tidak bisa diakses langsung dari luar class  
    private String nomorRekening;  
    private String pemilik;  
    private double saldo;  
  
    // 3. Konstruktor: memastikan setiap objek RekeningBank  
    // langsung punya nomor rekening, pemilik, dan saldo awal yang valid  
    // saat pertama kali dibuat  
    public RekeningBank(String nomorRekening, String pemilik, double saldoAwal) {  
        if (saldoAwal < 0) {  
            throw new IllegalArgumentException("Saldo awal tidak boleh negatif");  
        }  
        this.nomorRekening = nomorRekening;  
        this.pemilik = pemilik;  
        this.saldo = saldoAwal;  
    }  
  
    // 2. Method khusus (getter) untuk membaca nilai atribut
```

```

public double getSaldo() {
    return saldo;
}

public String getPemilik() {
    return pemilik;
}

// 2. Method khusus (setter/behavior) untuk mengubah nilai atribut,
// lengkap dengan validasi
public void setor(double jumlah) {
    if (jumlah <= 0) {
        throw new IllegalArgumentException("Jumlah setoran harus positif");
    }
    saldo += jumlah;
}

public void tarik(double jumlah) {
    if (jumlah <= 0) {
        throw new IllegalArgumentException("Jumlah penarikan harus positif");
    }
    if (jumlah > saldo) {
        throw new IllegalStateException("Saldo tidak mencukupi");
    }
    saldo -= jumlah;
}
}

```

Kenapa saldo harus private? Kalau saldo dibuat public, siapa pun bisa langsung menulis `rekening.saldo = 999999999`; tanpa lewat proses transaksi yang sah — itu jelas berbahaya. Dengan enkapsulasi, satu-satunya cara mengubah saldo adalah lewat `setor()` atau `tarik()`, yang sudah dilengkapi validasi (misalnya tidak boleh menarik melebihi saldo, tidak boleh setor jumlah negatif, dsb).

Contoh Kasus: Kapan Harus Membuat Konstruktor

Konstruktor dibutuhkan ketika **sebuah objek tidak boleh ada dalam keadaan "kosong" atau tidak lengkap** saat pertama kali dibuat. Contoh lain selain rekening bank:

Objek Mobil — setiap mobil di dunia nyata pasti sudah punya nomor plat, merek, dan tahun produksi sejak "lahir" dari pabrik. Tidak masuk akal ada objek Mobil tanpa nomor plat sama sekali.

```

public class Mobil {
    private String nomorPlat;
    private String merek;
    private int tahunProduksi;

    public Mobil(String nomorPlat, String merek, int tahunProduksi) {
        this.nomorPlat = nomorPlat;
        this.merek = merek;
        this.tahunProduksi = tahunProduksi;
    }
    // getter/setter di sini...
}

```

Tanpa konstruktor, kita berisiko membuat objek Mobil yang atributnya masih null atau 0, padahal secara logis dunia nyata itu tidak valid — mobil harus punya identitas lengkap sejak awal dibuat.

```

PS C:\Users\zyrex Pro\OneDrive\Documents\Kelas 2\Matkul Sem 3\PBO> cd "c:\Users\zyrex Pro\OneDrive\Documents\Kelas 2\Matkul Sem 3\PBO\Minggu 3\" ; if ($?) { javac TestNasabahParameter.java } ; if ($?) { java TestNasabahParameter }
Setor Rp250000
=== DATA NASABAH ===
Nomor Rekening : 987654321
Nama           : Carlos Rama
Saldo Awal    : 250000
Setor Rp50000
Saldo Setelah Setor : 300000
Tarik Rp100000
Saldo Setelah Tarik : 200000
PS C:\Users\zyrex Pro\OneDrive\Documents\Kelas 2\Matkul Sem 3\PBO\Minggu 3>

```

```

=== STATUS MOBIL ===
Kontak : OFF
Kecepatan : 0 km/jam

Kecepatan tidak bisa bertambah karena mesin OFF!
Mesin dinyalakan.
Kecepatan bertambah 10 km/jam.
Kecepatan bertambah 10 km/jam.
Kecepatan bertambah 10 km/jam.
=== STATUS MOBIL ===
Kontak : ON
Kecepatan : 30 km/jam

Kecepatan berkurang 10 km/jam.
Kecepatan berkurang 10 km/jam.
=== STATUS MOBIL ===
Kontak : ON
Kecepatan : 10 km/jam

Mesin dimatikan.
=== STATUS MOBIL ===
Kontak : OFF
Kecepatan : 0 km/jam

```